Partie 1: manipuler des données

Lino Galiana
Version du 2 novembre 2026

`Python` s'est imposé comme une alternative très crédible à `R` dans la manipulation de données. L'écosystème `Pandas` a permis de démocratiser l'utilisation des DataFrames dans `Python` et faciliter la manipulation de données structurées grâce à la philosophie `SQL`. `Python` reste également le langage le plus pratique pour récupérer et manipuler des données moins structurées (*webscraping*, API). `Python` tend à devenir, grâce à son écosystème dynamique, le langage *« one to rule them all »*.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

Résumé de cette partie **2**
Exercices **2**
Pour aller plus loin **3**
Références 3
Informations additionnelles **3**
Bibliographie **5**

Si on associe souvent les *data scientists* à la mise en oeuvre de modèles d'intelligence artificielle, il est important de ne pas oublier que l'entraînement et l'utilisation de ces modèles ne représente pas forcément le quotidien des *data scientists*.

En pratique, la récupération de sources de données hétérogènes, la structuration et harmonisation de celles-ci en vue d'une analyse exploratoire préalable à la modélisation ou la visualisation représente une part importante du travail des *data scientists*. Dans de nombreux environnements c'est même l'essence du travail du *data scientist*. L'élaboration de modèles pertinents requiert en effet une réflexion approfondie sur les données ; une étape que l'on ne saurait négliger.

Ce cours, comme de nombreuses ressources introductives sur la *data science* [1], [2], [3], proposera donc beaucoup d'éléments sur la manipulation de données, compétence essentielle pour les *data scientists*.

Les logiciels de programmation orientés autour du concept de base de données sont devenus les outils principaux des *data scientists*. Le fait de pouvoir appliquer un certain nombre d'opérations standards sur des bases de données, quelle que soit leur nature, permet aux programmeurs d'être plus efficaces que s'ils devaient répéter ces opérations à la main, comme dans `Excel`.

Tous les langages de programmation dominants dans l'écosystème de la *data science* reposent sur le principe du *dataframe*. Il s'agit même d'un objet central dans certains logiciels, notamment `R`. La logique `SQL`, un langage de déclaration d'opérations sur des données qui a déjà plus de cinquante ans, offre un cadre pertinent pour effectuer des opérations standardisées sur les colonnes (création de nouvelles colonnes, sélection de sous-ensemble de lignes...).

Néanmoins, le *dataframe* ne s'est imposé que récemment en `Python`, grâce au package `Pandas` créé par Wes McKinney. L'essor de la librairie `Pandas` (téléchargée plus de 5 millions de fois par jour en 2023) est pour beaucoup dans le succès de `Python` dans l'écosystème de la *data science* et a amené, en quelques années, a un renouvellement complet de la manière de coder en `Python`, ce langage si malléable, autour de l'analyse de données.

Cette partie du cours est une introduction générale à l'écosystème très riche de la manipulation de données avec `Python`. Ces chapitres évoquent aussi bien la récupération de données que la restructuration et la production d'analyse à partir de celles-ci.

Résumé de cette partie

`Pandas` est devenu incontournable dans l'écosystème `Python` pour la *data science*. `Pandas` est lui-même construit à partir du package `Numpy`, qu'il est utile de comprendre pour être à l'aise avec `Pandas`. `Numpy` est une librairie bas-niveau pour stocker et manipuler des données. `Numpy` est au coeur de l'écosystème de la *data science* car la plupart des librairies, même celles qui manient des objets destructurés, utilisent des objets construits à partir de `Numpy`[1].

L'approche `Pandas`, qui offre un point d'entrée harmonisé pour manipuler des jeux de données de nature très différente, a été étendue aux objets géographiques avec `Geopandas`. Il est ainsi possible de manipuler des données géographiques comme s'il s'agissait de données structurées classiques. Les données géographiques et la représentation cartographique deviennent de plus en plus commun avec la multiplication de données ouvertes localisées et de *big-data* géolocalisées.

Cependant, les données structurées, importées depuis des fichiers plats ne représentent pas l'unique source de données. Les API et le *webscraping* permettent de télécharger ou d'extraire des données de manière très flexible depuis des pages web ou des guichets spécialisés. Ces données, notamment celles obtenues par *webscraping* nécessitent souvent un peu plus de travail de nettoyage de données, notamment des chaînes de caractère.

L'écosystème `Pandas` représente donc un couteau-suisse pour l'analyse de données. C'est pour cette raison que ce cours développera beaucoup de contenu dessus. Avant d'essayer de mettre en oeuvre une solution *ad hoc*, il est souvent utile de se poser la question suivante : *« ne pourrais-je pas le faire avec les fonctionnalités de base de `Pandas` ? »* Se poser cette question peut éviter des chemins ardus et faire économiser beaucoup de temps.

Néanmoins, `Pandas` n'est pas adapté à des données ayant une volumétrie importante. Pour traiter de telles données, il est plutôt recommandé de privilégier `Polars` ou `Dask` qui reprennent la logique de `Pandas` mais optimisent son fonctionnement, `Spark` si on a une infrastructure adaptée, généralement dans des environnements *big data*, ou `DuckDB` si on est prêt à utiliser des requêtes SQL plutôt qu'une librairie haut-niveau.

Exercices

Cette partie présente à la fois des tutoriels détaillés et des exercices guidés. Il est possible de les consulter sur ce site ou d'utiliser l'un des badges présents en début de chapitre, par exemple ceux-ci pour ouvrir le chapitre d'exercices sur `Pandas`:

[1] Certaines librairies commencent, petit à petit, à s'émanciper de `Numpy` qui n'est pas toujours le plus adapté pour la gestion de certains types de données. Le *framework* `Arrow` tend à devenir la couche basse utilisée par de plus en plus de librairies de *data science*. Ce *post* de blog approfondit de manière très pédagogique ce sujet.

Pour aller plus loin

Ce cours n'aborde pas vraiment les questions de volumétrie ou de vitesse de calcul. `Pandas` peut montrer ses limites dans ce domaine sur des jeux de données d'une volumétrie conséquente (plusieurs Gigas).

Il est ainsi intéressant de porter attention à:

- Le livre Modern Pandas pour obtenir des éléments supplémentaires sur la question de la performance avec `Pandas` ;
- La question des objets *sparse* ;
- Les *packages* `Dask` ou `Polars` pour accélérer les calculs ;
- `DuckDB` pour effectuer de manière très efficace des requêtes SQL ;
- `PySpark` pour des données très volumineuses.

Références

Voici une bibliographie sélective des ouvrages intéressants en complément des chapitres de la partie « Manipulation » de ce cours :

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
```

```
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie

[1] H. Wickham, M. Çetinkaya-Rundel, et G. Grolemund, *R for data science.* " O'Reilly Media, Inc.", 2023.

[2] J. VanderPlas, *Python data science handbook: Essential tools for working with data.* " O'Reilly Media, Inc.", 2016.

[3] W. McKinney, *Python for data analysis: Data wrangling with Pandas, NumPy, and IPython.* " O'Reilly Media, Inc.", 2012.